RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

CS3508 – Cloud Security and Migration
Project Case Study Report

School of Computer Science and Engineering
2026

Blink Access- Just-In-Time Temporary AWS Credential Vending System

Submitted in partial fulfillment for the award of the degree of

BTech(Hons.) in Computer Science and Engineering

By

Aneesh Adithya (1RVU23CSE054)

Pratham RD (1RVU23CSE348)

Varun UB (1RVU23CSE530)

Guided by
Prof. Aparna R

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

CERTIFICATE

Certified that the CS3508 Advanced AWS Cloud computing Project Case Study titled “Blink Access- Just-In-Time Temporary AWS Credential Vending System” is carried out by Aneesh Adithya SR(1RVU23CSE054),Pratham RD(1RVU23CSE348), Varun U Baduger1RVU23CSE530), who are bonafide students of the School of Computer Science and Engineering, RV University, Bengaluru, during the year 2025–26. It is certified that all corrections/ suggestions from all the continuous internal evaluations have been incorporated in the project and in this report.

Faculty Guide	Program Director
Prof. Aparna R	Dr. K C Narendra

ABSTRACT

BlinkAccess is a Just-In-Time (JIT) credential vending system built on AWS that addresses a fundamental cloud security problem — permanent AWS credentials. Traditional IAM models grant developers standing access to cloud resources, which creates risks such as credential leakage, over-provisioned access, and difficulty in auditing.

BlinkAccess solves this by implementing a Zero Standing Privileges model. Users authenticate via Amazon Cognito and request temporary, scoped AWS credentials through a React-based web portal. The system uses AWS STS to issue time-bound credentials tied to specific IAM roles. Every request is logged in DynamoDB, email alerts are sent via SNS, and expired credentials are automatically revoked by a CloudWatch-triggered Lambda function.

The system supports three resource types — Amazon S3 (write to bucket), DynamoDB (read table), and EC2 (describe instances) — with configurable durations of 15, 30, or 60 minutes. The entire project is built on AWS free tier services including Cognito, Lambda, API Gateway, DynamoDB, SNS, STS, IAM, and CloudWatch.

BlinkAccess demonstrates that enterprise-grade Zero Standing Privilege architecture is achievable using serverless AWS services, making it a cost-effective and scalable solution for credential management in modern cloud environments.

INTRODUCTION

Introduction

With the rapid growth of cloud-based applications, managing access to cloud resources securely has become increasingly important. Organizations give developers permanent AWS credentials — access keys that never expire. While convenient, this model creates serious security risks. A leaked credential gives an attacker permanent access. When an employee leaves, permissions must be manually revoked across multiple services. Auditing becomes nearly impossible because everyone has standing access at all times.

Just-In-Time (JIT) access solves this by replacing permanent standing access with dynamic, time-bound credentials. Users request access only when needed, receive credentials scoped to a specific resource for a limited duration, and those credentials expire automatically with no manual intervention required.

BlinkAccess is our implementation of this principle — a full-stack web application built on AWS that enables authenticated users to request temporary AWS credentials, use them to access specific resources, and have them revoked automatically when the duration expires. The name reflects the core concept: access appears in a blink and disappears just as quickly.

Problem Statement

Cloud environments face persistent security challenges when relying on permanent IAM credentials:

- Credential sprawl — access keys stored in laptops, config files, and CI/CD pipelines
- No expiry — permanent credentials remain valid unless manually revoked
- Over-provisioning — developers given broad access for convenience
- Poor auditability — no clear record of who accessed what and when
- Offboarding risk — departing employees may retain access indefinitely
- High blast radius — a single leaked credential can compromise entire AWS account

There is a need for a secure, automated, and auditable access management system that:

- Issues credentials only when explicitly requested
- Scopes credentials to the minimum required permissions
- Automatically expires access after a defined duration
- Logs every access request for compliance and auditing
- Requires zero manual revocation

Objectives

The main goals of this project are:

Primary Objective

- Build a Just-In-Time credential vending system on AWS
- Implement Zero Standing Privileges — no permanent access for any user
- Issue temporary, scoped AWS credentials via STS AssumeRole
- Automatically revoke expired credentials without human intervention

Secondary Objectives

- Provide a user-friendly web interface for requesting access
- Maintain a complete audit trail of all access requests
- Send real-time email alerts on every grant and revocation
- Implement AWS security best practices throughout
- Build entirely within AWS free tier

Scope of the Project

This project covers the design and implementation of a complete JIT access management system. The scope includes:

- User authentication using Amazon Cognito with JWT-based identity verification
- REST API backend using API Gateway and Python Lambda functions
- Temporary credential issuance via AWS STS AssumeRole
- Access policy enforcement using DynamoDB-stored rules
- Automated expiry and revocation using CloudWatch scheduled rules
- Real-time notifications using Amazon SNS
- React frontend with live credential display and countdown timer
- CLI-based credential testing and verification

TOOLS AND TECHNOLOGIES

Core Security Services

Amazon Cognito

What it is: AWS's managed user authentication service. Handles sign-up, login, identity verification, and token issuance.

Role in BlinkAccess: Manages the login page for BlinkAccess. When a user enters their credentials, Cognito verifies their identity and issues a JWT (JSON Web Token). This token is attached to every API request, ensuring only verified users can request temporary credentials.

AWS Lambda

What it is: A serverless compute service. Runs code on demand without managing servers — you pay only when the function executes.

Role in BlinkAccess: Three Lambda functions power the entire backend. request_access issues temporary credentials via STS. list_requests fetches the user's access history from DynamoDB. revoke_expired scans DynamoDB every 5 minutes and marks expired grants as EXPIRED.

Amazon API Gateway

What it is: A managed service that creates, publishes, and manages REST APIs. Acts as the entry point for HTTP requests to the backend.

Role in BlinkAccess: Exposes two endpoints — POST /request (triggers request_access Lambda) and GET /requests (triggers list_requests Lambda). The React frontend communicates exclusively through API Gateway, which routes requests to the appropriate Lambda function.

AWS STS (Security Token Service)

What it is: AWS service that issues temporary, short-lived credentials by assuming IAM roles. The issued credentials are cryptographically signed and time-limited.

Role in BlinkAccess: The core of BlinkAccess. When a user requests access, the request_access Lambda calls STS AssumeRole with a specific scoped IAM role. STS returns temporary AccessKeyId, SecretAccessKey, and SessionToken valid for the requested duration only.

Amazon DynamoDB

What it is: AWS's fully managed NoSQL database. Serverless, fast, and highly scalable with no infrastructure management required.

Role in BlinkAccess: Two tables serve different purposes. AccessRequests logs every credential grant with userId, resource type, duration, status (ACTIVE/EXPIRED), createdAt, and expiresAt. AccessPolicies stores the allowed resources and their maximum permitted durations.

Amazon SNS (Simple Notification Service)

What it is: A managed pub/sub messaging service that delivers notifications to subscribers via email, SMS, or other protocols.

Role in BlinkAccess: Sends real-time email alerts to the administrator. Every time a user is granted access (via request_access Lambda) or access is auto-revoked (via revoke_expired Lambda), SNS publishes a message to the JITAccessAlerts topic which delivers it to the subscribed Gmail address.

Amazon CloudWatch

What it is: AWS's monitoring and observability service with scheduling capabilities. Tracks metrics, logs, and can trigger automated actions on a schedule.

Role in BlinkAccess: Runs a scheduled rule every 5 minutes that triggers the revoke_expired Lambda. This automated trigger is what enables credential auto-expiry without any human involvement — CloudWatch is the heartbeat of BlinkAccess's revocation system.

IAM (Identity and Access Management)

What it is: AWS's permission management service. Controls who can do what across all AWS services using roles and policies.

Role in BlinkAccess: Three tightly scoped IAM roles define the boundaries of each credential type. S3WriteOnlyRole allows only PutObject and GetObject on a specific bucket. DynamoReadOnlyRole allows only GetItem and Query on one table. EC2ReadOnlyRole allows only DescribeInstances and DescribeSecurityGroups. STS issues credentials tied to exactly one of these roles per request.

Methodology

The process of implementing the automated cloud forensics and incident response system consists of deploying a cloud-based architecture that monitors activities, detects suspicious events, collects forensic evidence, and stores it securely. The methodology is divided into the following steps:

Step 1: Environment Setup

- The system is built using Amazon Web Services
- Required services such as EC2, S3, IAM, CloudTrail, EventBridge, Lambda, and SSM are configured
- IAM roles are created to provide secure access between services
- This step establishes the foundation of the system

Step 2: Launching EC2 Instance

- An EC2 instance is created to act as the target server
- The instance is configured without SSH access for better security
- SSM Agent is enabled to allow remote command execution
- This instance acts as the source for forensic data collection

Step 3: Enabling CloudTrail Logging

- CloudTrail is enabled to record all API activities in the AWS account
- It captures details such as user identity, action performed, time, and IP address
- Logs are stored in an S3 bucket for auditing
- This step ensures complete visibility of all actions

Step 4: Event Detection using EventBridge

- EventBridge is configured with rules to detect suspicious activities
- It monitors CloudTrail events continuously
- Specific events include:

  StopInstances

  TerminateInstances

  CreateUser

  AttachUserPolicy

  AuthorizeSecurityGroupIngress
- When a match is found, it triggers the next process automatically

Step 5: Automated Response using Lambda

- AWS Lambda function is created to handle incident response
- It is triggered automatically by EventBridge
- The function processes event details such as action, user, and time

- It acts as the central controller of the system

Step 6: Forensic Data Collection using SSM

- Lambda uses Systems Manager (SSM) to run commands on EC2
- Commands executed include:
  ps aux (running processes)
  netstat - tulpn (network connections)
  who (current users)
  last (login history)
- This step collects real-time evidence from the server

Step 7: Storing Evidence in S3

- All collected forensic data is stored in Amazon S3
- Data is saved in structured JSON format with timestamps
- S3 ensures secure and durable storage
- Evidence can be accessed later for investigation

Step 8: Monitoring and Logging

- Lambda execution logs are stored in CloudWatch
- Helps track system activity and debug errors
- Ensures transparency of the entire workflow

Step 9: System Testing

- Suspicious activities are simulated (e.g., stopping EC2 instance)
- The system is tested end-to-end
- Verification is done to ensure automatic detection and evidence collection

Step 10: Continuous Improvement

- System performance is monitored regularly
- Detection rules can be updated based on new threats
- Helps improve accuracy and reliability of the system

System Architecture Diagram

System Architecture Explanation

The architecture of the PoLP Enforcer - Just-in-Time Access Manager is designed to enforce the Principle of Least Privilege by issuing temporary, scoped AWS credentials on demand, automatically revoking them after expiry, and maintaining a complete audit trail of all access activity using Amazon Web Services.

The complete workflow is explained step-by-step as follows:

Step 1: User Authentication (Cognito)

- A developer or team member opens the web application hosted on AWS Amplify. The user is required to log in before accessing any feature of the system.
- Amazon Cognito handles the authentication process
- The user enters their email and password on the Cognito-powered login screen
- Upon successful login, Cognito issues a JWT token to the browser
- This token proves the user's identity and is attached to every API request made by the frontend
- Unauthenticated users are completely blocked from submitting any access request

Step 2: Access Request Submission (React Frontend + API Gateway)

- After logging in, the user fills out an access request form in the React web application. The user selects the resource they need access to and the duration for which they need it.
- Resource options include: S3 bucket write access, DynamoDB read access, EC2 describe access
- Duration options include: 15 minutes, 30 minutes, or 60 minutes
- The frontend sends a POST request to API Gateway with the user ID, resource type, and duration
- The Cognito JWT token is attached in the Authorization header
- API Gateway validates the token using the Cognito Authorizer before allowing the request to proceed
- If the token is missing or invalid, the request is rejected immediately

Step 3: Policy Validation and Credential Generation (Lambda + DynamoDB + STS)

- The API Gateway triggers the request_access Lambda function. This function acts as the central controller of the entire system.
- Lambda first reads the AccessPolicies DynamoDB table to fetch the rules for the requested resource
- It checks whether the requested duration is within the allowed maximum for that resource type
- If the duration exceeds the policy limit, the request is rejected and an error is returned to the frontend
- If the request is valid, Lambda calls AWS STS using the AssumeRole API
- STS generates a temporary set of credentials: an AccessKeyId, SecretAccessKey, and SessionToken
- These credentials are scoped strictly to the IAM role defined for the requested resource
- The credentials are valid only for the exact duration the user requested and automatically expire at the AWS infrastructure level

Step 4: Grant Storage and Audit Logging (DynamoDB)

- Every access request, whether approved or rejected, is recorded in the system.
- Lambda writes a new record to the AccessRequests DynamoDB table
- The record contains the request ID, user ID, resource type, duration, status (ACTIVE), creation timestamp, and expiry timestamp
- Only the AccessKeyId is stored - the SecretAccessKey and SessionToken are never persisted for security reasons
- This table forms the complete audit trail of all access grants in the system
- The frontend reads this table via the list_requests Lambda to display the user's access history

Step 5: Real-Time Alert to Admin (SNS)

- As soon as a grant is issued, the administrator is notified immediately.
- Lambda publishes a message to the SNS topic named JITAccessAlerts
- The message contains the user ID, resource type, duration, and exact expiry time
- SNS delivers this message as an email to the subscribed administrator address
- This ensures the admin has full real-time visibility into every access event without needing to check the console manually
- A separate SNS alert is also sent when credentials are revoked, completing the notification lifecycle

Step 6: Credential Display and Countdown (React Frontend)

- The temporary credentials are returned to the frontend and displayed to the user securely.
- The React application displays the AccessKeyId, a masked SecretAccessKey, and a truncated SessionToken
- A live countdown timer starts immediately, showing the user exactly how many seconds remain before their access expires
- The user can use these credentials in the AWS CLI or SDK to perform only the specific actions the scoped role allows
- When the timer reaches zero, the credentials section disappears from the UI automatically

Step 7: Automatic Revocation (CloudWatch + Lambda)

- The system does not rely on users to return access manually. Revocation is fully automated.
- A CloudWatch Events rule is configured to fire every 5 minutes on a fixed schedule
- This rule triggers the revoke_expired Lambda function automatically
- The Lambda scans the AccessRequests table for any records where the status is ACTIVE and the expiry time is in the past
- For each expired grant found, the status is updated to EXPIRED in DynamoDB
- An SNS revocation alert is sent to the admin listing all revoked grant IDs
- The STS credentials themselves also expire independently at the AWS infrastructure level, providing a second layer of enforcement

Step 8: Permission Management using IAM

- AWS IAM controls and restricts communication between every service in the system.
- The LambdaJITExecutionRole allows Lambda to call STS, read and write DynamoDB, publish to SNS, and write to CloudWatch Logs
- The three scoped roles - S3WriteOnlyRole, DynamoReadOnlyRole, and EC2ReadOnlyRole - each contain only the minimum permissions required for their resource

- API Gateway is permitted to invoke Lambda functions only
- Cognito is permitted only to authenticate users and issue tokens
- No service has access beyond what it strictly needs, implementing Defense in Depth across the entire architecture

Overall Working

- The system operates in a fully automated and secure manner:
- A user logs in through Cognito and submits a time-bound access request via the React frontend
- API Gateway validates the identity and routes the request to Lambda
- Lambda enforces policy rules, calls STS to generate scoped temporary credentials, and logs the grant to DynamoDB
- SNS notifies the admin instantly via email
- The user receives real-time credentials with a countdown timer in the browser
- CloudWatch automatically triggers revocation Lambda every 5 minutes to expire stale grants
- Every action is logged in DynamoDB and CloudWatch, providing a tamper-evident audit trail
- No permanent credentials are ever issued, and no human intervention is required for revocation

ADVANTAGES OF THE SYSTEM

Advantages of the System

Strict Enforcement of Least Privilege:

The system ensures that no user ever holds permanent or broad AWS permissions. Every credential issued is scoped to the exact resource and action required, directly implementing the Principle of Least Privilege at the infrastructure level using IAM and STS.

Automatic Credential Expiry:

Temporary credentials issued via AWS STS automatically expire at the exact time specified by the user. Even if credentials are stolen or leaked, they become completely useless after the expiry window — significantly reducing the blast radius of any security incident.

Zero Permanent Credentials:

The system never issues or stores permanent IAM access keys. All access is granted through short-lived STS session tokens, eliminating the risk of credential sprawl, forgotten API keys, and long-term exposure.

Fully Automated Revocation:

The CloudWatch-triggered revocation Lambda runs every five minutes and automatically marks expired grants without any human intervention. Administrators do not need to manually track or cancel access — the system handles the entire lifecycle end to end.

Real-Time Admin Visibility:

Every grant and revocation triggers an instant SNS email alert to the administrator. Admins are always aware of who has access to what resource and for how long, without needing to open the AWS console or query any database manually.

Complete Audit Trail:

Every access request, approval, credential generation, and revocation is recorded in DynamoDB with timestamps. This provides a tamper-evident log of all activity in the system, which is essential for forensic investigation and compliance reporting.

Policy-Driven Access Control:

Access rules are stored in the AccessPolicies DynamoDB table rather than hardcoded in application logic. This means new resource types can be added and maximum durations can be adjusted without touching any code — just update the table.

Identity-Gated Access:
All access requests are authenticated through Amazon Cognito before reaching the backend. The API Gateway Cognito Authorizer ensures anonymous or unauthorized requests are rejected at the entry point, before any Lambda function is even invoked.

Reduced Attack Surface:
Because users receive only time-limited, resource-specific credentials, a compromised account can only damage a narrow, predefined scope of resources for a short window of time. This directly limits the damage an attacker can cause even with valid credentials.

Serverless and Cost-Effective:
The entire backend runs on AWS Lambda, DynamoDB, API Gateway, SNS, and CloudWatch — all of which operate within the AWS Free Tier for the scale of a college project. There are no servers to manage, no idle compute costs, and no infrastructure maintenance required.

Scalable Without Redesign:
New resource types such as RDS access, Lambda invocation, or Secrets Manager reads can be added simply by creating a new IAM role and inserting a new item into the AccessPolicies table. The core system architecture does not need to change as the scope grows.

Supports Defense in Depth:
The system applies multiple independent security layers — Cognito for identity, API Gateway for request validation, IAM for permission boundaries, STS for credential scoping, CloudWatch for automated revocation, and SNS for monitoring. If any single layer fails, the others continue to provide protection..

CONCLUSION AND FUTURE SCOPE

Conclusion

- The proposed system successfully implements a Just-in-Time Access Manager that enforces the Principle of Least Privilege using Amazon Web Services, ensuring that no user ever holds permanent or excessive cloud permissions.
- It provides a fully functional React web application where authenticated users can request temporary, scoped AWS credentials for specific resources and durations, making the concept of least privilege practical and usable in real-world scenarios.
- The system leverages AWS Cognito for identity verification, ensuring that only authenticated users can initiate access requests through the secured API Gateway endpoint.
- Temporary credentials are generated using AWS STS AssumeRole, which scopes access to the exact resource and action required, and automatically expires the credentials at the AWS infrastructure level without any manual intervention.
- All access grants, policy rules, and revocation events are recorded in Amazon DynamoDB, providing a complete and tamper-evident audit trail suitable for security investigation and compliance reporting.
- The automated revocation mechanism powered by CloudWatch Events and Lambda ensures that expired grants are detected and marked every five minutes, eliminating the risk of forgotten or lingering access.
- Real-time SNS email alerts keep administrators informed of every grant and revocation, providing continuous visibility into the access activity of the system without requiring manual monitoring.
- Overall, the project demonstrates how cloud-native services can be combined to build a practical, automated, and policy-driven access control system that directly addresses real-world security threats such as credential sprawl, privilege creep, and long-term credential exposure.

Future Scope

- The system can be extended to support an approval workflow where high-risk resource requests such as RDS or IAM access require explicit admin approval before STS credentials are issued, adding a human-in-the-loop layer for sensitive operations.
- Multi-Factor Authentication can be enforced through Cognito for users requesting access to critical resources, adding an additional identity verification step beyond username and password.
- Machine Learning-based anomaly detection can be integrated to identify unusual access patterns, such as a user requesting access to multiple resource types within a short time window, and automatically flag or block such requests.
- The system can be expanded to support a real-time admin dashboard that shows all active grants, time remaining on each credential, user activity history, and revocation events in a single view, improving operational visibility.
- Integration with AWS Organizations can allow the system to manage Just-in-Time access across

multiple AWS accounts, making it suitable for enterprise environments with complex multi-account setups.

- The notification system can be enhanced beyond SNS email to include SMS alerts via SNS, Slack webhook notifications, or integration with PagerDuty for on-call incident management teams.
- The system can be integrated with Security Information and Event Management tools such as Splunk or AWS Security Hub to correlate JIT access events with other security signals across the organization.
- Support for session recording can be added using AWS CloudTrail data events, capturing every API call made using the temporary credentials and storing a detailed record of exactly what the user did during their access window.
- The access request interface can be extended with a mobile-friendly progressive web app, allowing developers to request and manage temporary access from their phones during on-call situations.
- Automated compliance reporting can be built on top of the DynamoDB audit trail, generating weekly or monthly reports showing access patterns, policy violations, and revocation statistics for auditors and security teams.

REFERENCES

- Amazon Web Services. (n.d.). *AWS Identity and Access Management – Access Control and Security Management.* Retrieved from https://aws.amazon.com/iam/
- Amazon Web Services. (n.d.). *AWS Security Token Service – Temporary Security Credentials.* Retrieved from https://aws.amazon.com/iam/features/temporary-credentials/
- Amazon Web Services. (n.d.). *Amazon Cognito – User Authentication and Access Control.* Retrieved from https://aws.amazon.com/cognito/
- Amazon Web Services. (n.d.). *AWS Lambda – Serverless Compute Service.* Retrieved from https://aws.amazon.com/lambda/
- Amazon Web Services. (n.d.). *Amazon DynamoDB – Managed NoSQL Database Service.* Retrieved from https://aws.amazon.com/dynamodb/
- Amazon Web Services. (n.d.). *Amazon API Gateway – Build, Deploy and Manage APIs.* Retrieved from https://aws.amazon.com/api-gateway/
- Amazon Web Services. (n.d.). *Amazon SNS – Simple Notification Service.* Retrieved from https://aws.amazon.com/sns/
- Amazon Web Services. (n.d.). *Amazon CloudWatch – Monitoring and Observability Service.* Retrieved from https://aws.amazon.com/cloudwatch/
- Amazon Web Services. (n.d.). *AWS Amplify – Build and Host Full Stack Web Applications.* Retrieved from https://aws.amazon.com/amplify/
- Amazon Web Services. (n.d.). *IAM Best Practices – Granting Least Privilege.* Retrieved from https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html
- Amazon Web Services. (n.d.). *Using IAM Roles – Delegate Access Across AWS Accounts.* Retrieved from https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
- Amazon Web Services. (n.d.). *AssumeRole API Reference – AWS Security Token Service.* Retrieved from https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRole.html
- Amazon Web Services. (n.d.). *Amazon Cognito User Pools – Authentication and Authorization.* Retrieved from https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-identity-pools.html
- Amazon Web Services. (n.d.). *AWS Lambda – How Lambda Works with API Gateway.* Retrieved from https://docs.aws.amazon.com/lambda/latest/dg/services-apigateway.html
- Saltzer, J. H., & Schroeder, M. D. (1975). *The Protection of Information in Computer Systems.* Proceedings of the IEEE, 63(9), 1278–1308. Retrieved from https://ieeexplore.ieee.org/document/1451869
- National Institute of Standards and Technology. (2020). *Zero Trust Architecture – NIST Special Publication 800-207.* Retrieved from https://doi.org/10.6028/NIST.SP.800-207
- Shu, X., Yao, D., & Bertino, E. (2015). *Privacy-Preserving Detection of Sensitive Data Exposure.* IEEE Transactions on Information Forensics and Security. Retrieved from https://ieeexplore.ieee.org/document/7047789

- Reactive Systems. (n.d.). *React – A JavaScript Library for Building User Interfaces.* Retrieved from https://react.dev/